

Python aplicado al análisis y procesamiento de datos

Facilitador: Javier Sánchez Padilla.

Entornos virtuales.

- Aislamiento.
- Portabilidad.
- Control de dependencias.

Entornos virtuales.

Creación del entorno virtual

```
Python -m venv <nombre_del_entorno>
```

Activar el entorno virtual

Linux: `source venv/bin/activate`

Windows: `venv\Scripts\activate`

(**venv** o el nombre asignado al entorno)

Al activar el entorno virtual el prompt cambiará indicando el nombre del entorno virtual creado.

Entornos virtuales.

Desactivación del entorno virtual

Linux Windows: deactivate

Visualizar paquetes instalados en nuestro entorno

Linux: pip3 freeze

Windows: pip freeze

Entornos virtuales.

Crear archivo de requerimientos

Linux: `pip3 freeze > requirements.txt`

Windows: `pip freeze > requirements.txt`

Instalar archivo de requerimientos

Linux: `pip3 install -r requirements.txt`

Windows: `pip install -r requirements.txt`

Ejecución de Visual Studio Code en nuestro entorno virtual.

FORMA 1. Ejecutar Visual Studio Code desde la terminal con el entorno virtual activado.

Visual Studio Code a menudo detecta el intérprete activo en la terminal en ejecución.

Linux: `code -r .`

Windows: `code -r .`

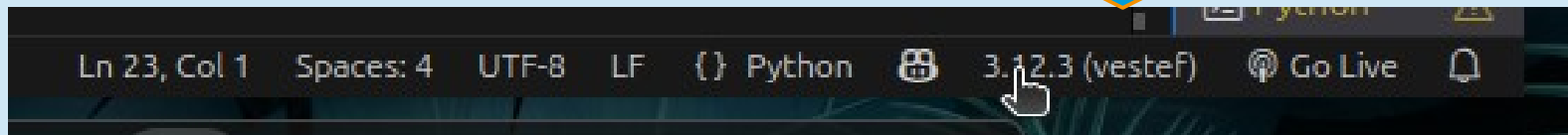
Ejecución de Visual Studio Code en nuestro entorno virtual.

FORMA 2. Esta es la manera recomendada para ejecutar el interprete de Python de nuestro entorno virtual.

- Abrimos Visual Studio Code y cargamos la carpeta de nuestro proyecto.
- Presiona **Ctrl + Shift + P** para abrir la paleta de comandos.
- Escribe "**Python: Select Interpreter**" y elige tu entorno virtual de la lista. .

VS Code buscará automáticamente los entornos virtuales que ha detectado en tu proyecto. Busca uno que diga `.venv` o `.\venv` (o la ruta a tu entorno virtual).

Selecciona el intérprete de tu entorno virtual. Verás el nombre del entorno virtual en la esquina inferior izquierda de la ventana de VS Code.



Ejercicio 01.

Crear dos entornos virtuales e instalar lo siguiente

En el primer entorno virtual instalar las librerías siguientes (pandas, djando, numpy).

En el segundo entorno virtual instale exactamente los mismos módulos que se instalaron en el primer entorno, pero además instale el siguiente módulo (pygame)

Para duplicar las librerías del primer entorno para instalar en el segundo entorno, se recomienda instalar a partir de un archivo de requerimientos desde el entorno uno y llevarlo al entorno dos para instalar.

¿Qué es PANDAS?

Pandas es una biblioteca de Python que se utiliza para trabajar con conjuntos de datos.

Tiene funciones para analizar, limpiar, explorar y manipular datos.

El nombre "Pandas" hace referencia tanto a "Panel Data" como a "Python Data Analysis" y fue creado por Wes McKinney en 2008.

¿Por qué utilizar Pandas?

Pandas nos permite analizar big data y sacar conclusiones basadas en teorías estadísticas.

- Los pandas (panel data) pueden limpiar conjuntos de datos desordenados y hacerlos legibles y relevantes.
- Los datos relevantes son muy importantes en la ciencia de datos.

¿Qué pueden hacer con pandas?

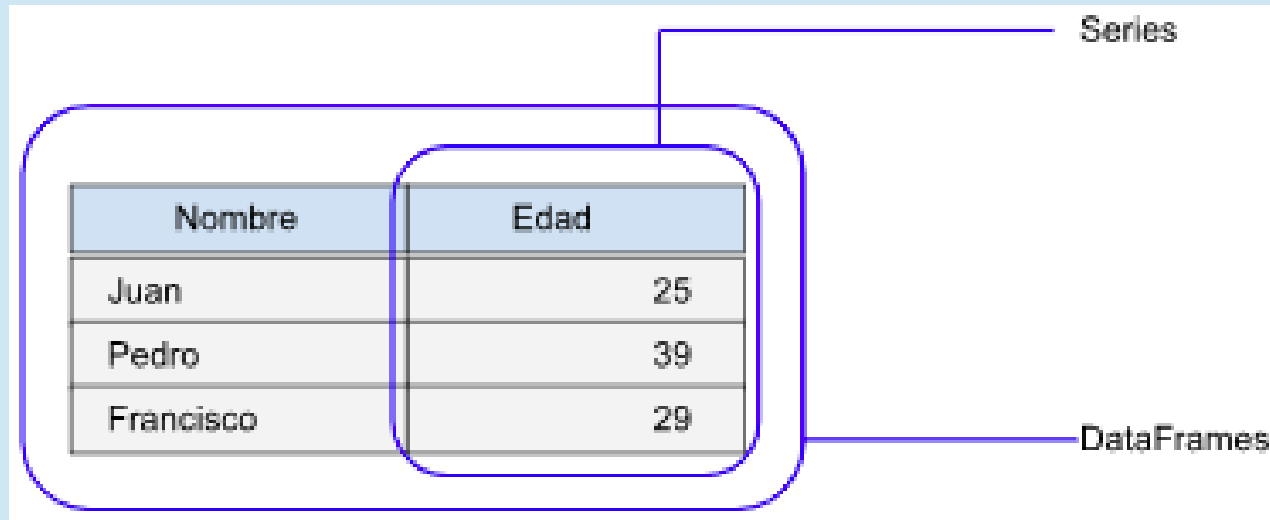
Pandas te da respuestas sobre los datos. Como:

- ¿Existe una correlación entre dos o más columnas?
 - ¿Cuál es el valor promedio?
 - ¿Valor máximo?
 - ¿Valor mínimo?
-
- Con pandas también pueden eliminar filas que no son relevantes o que contienen valores incorrectos, como valores vacíos o NULL. A esto se le llama limpiar los datos.

Pandas

Con esta librería incorporamos dos nuevos tipos de datos que son la estructura principal del análisis que son:

- Data Frames
- Series



Data frame.

	nombre	edad	
0	pedro	25	← Como vemos un dataframe es como
1	Juan	39	← una hoja de cálculo con filas y columnas
2	lorena	20	← Por eso a pandas se le conoce como el
			← Excel de python

`<class 'pandas.core.frame.DataFrame'>` ← Pertenece a la clase Pandas core Dataframe

Data frame.

Las series son arreglos de una dimensión y que tienen etiquetas para sus elementos.

En este caso el dataframe contiene dos series, la primera columna corresponde al indicador del índice, la segunda es la serie de nombre y la tercera es la serie de la edad, para acceder a ellas hacemos lo siguiente:

Sucursal
Centro
Sur
Norte
Este
Oeste

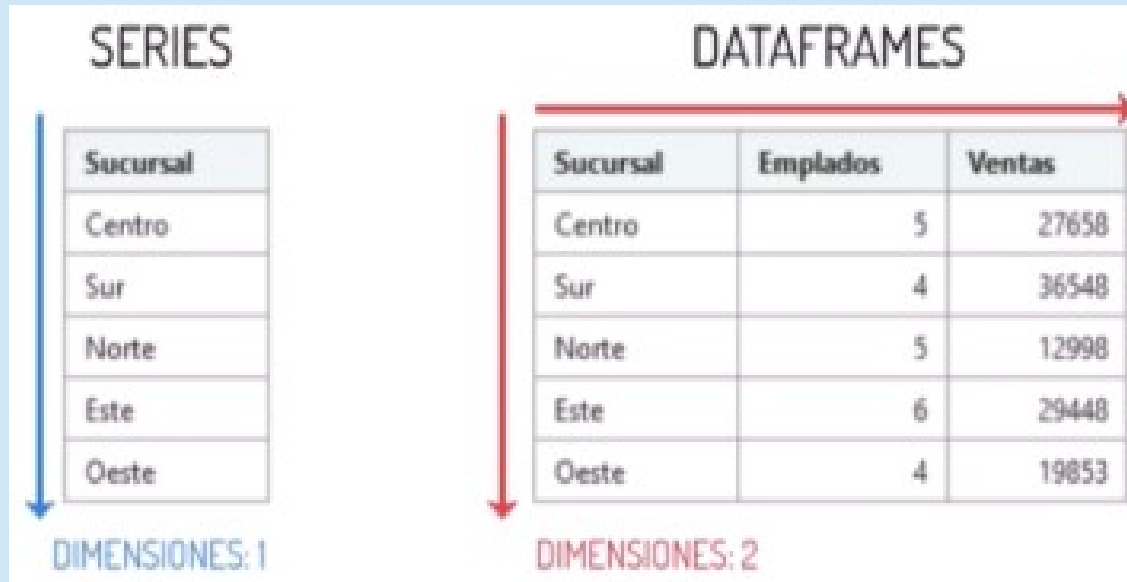
DIMENSIONES: 1

Sucursal	Emplados	Ventas
Centro	5	27658
Sur	4	36548
Norte	5	12998
Este	6	29448
Oeste	4	19853

DIMENSIONES: 2

Data frame.

un dataframe tiene tres atributos principales que son índices, filas y sus columnas que son sus valores, en cambio las series solo tienen dos de sus atributos que son el índice o filas y sus valores.



Ejercicio 1.

Utilizando la biblioteca Pandas, crea un DataFrame llamado **df_empleados** que contenga dos columnas: nombre y edad.

La columna nombre debe contener los nombres de tres empleados: 'Ana', 'Luis' y 'Carlos'

La columna edad debe contener las edades correspondientes: 30, 25 y 40

Finalmente, muestra el DataFrame df_empleados utilizando la función print().

Ejercicio 2.

Dado el DataFrame `df_empleados` creado en el ejercicio anterior:

La columna `nombre` debe contener los nombres de tres empleados: 'Ana', 'Luis' y 'Carlos'

La columna `edad` debe contener las edades correspondientes: 30, 25 y 40

Utiliza el método adecuado para seleccionar solo la columna `edad` y almacénala en una variable llamada `edades`.

Luego, muestra el contenido de `edades` utilizando la función `print()`.

Asegúrate de demostrar que el tipo de dato de `edades` es una Serie de Pandas. Utilizando la función `type()`.

Ejercicio 3

Dado el DataFrame `df_empleados` creado en el ejercicio anterior: La columna `nombre` debe contener los nombres de tres empleados: 'Ana', 'Luis' y 'Carlos', la columna `edad` debe contener las edades correspondientes: 30, 25 y 40

Para explorar sus atributos principales: `shape`, `columns`, y `index`.

Utilizando las siguientes variables respectivamente: `shape_df`, `columns_df`, `index_df`

Imprime los resultados de cada exploración para demostrar tu comprensión de la estructura de los DataFrames y las Series en Pandas.

El resultado esperado es el siguiente:

```
(3, 2)
```

```
Index('nombre', 'edad'), dtype='object')
```

```
RangeIndex(start=0, stop=3, step=1)
```

No se preocupe si en este momento no entiendes del todo estos resultados, recordemos que solo estamos explorando

Data frames en pandas.

```
variable = pandas.read_csv('precipitaciones.csv')
```

HEAD(). Con la función head() podemos ver las primeras cinco líneas del archivo para ver cómo se compone el archivo sin necesidad de tener que visualizar todo el archivo (n).

TAIL(). El método tail (cola) nos permite conocer las últimas cinco líneas del archivo (n).

Atributos y métodos de pandas.

SHAPE. Shape no es un método es un atributo y permite conocer el número de filas y columnas de un DataFrame.

COLUMNS. En Pandas, columns es un atributo de un DataFrame que devuelve un objeto Index que contiene los nombres de las columnas del DataFrame. Esto nos permitirá identificar de manera más sencilla las series.

INFO (). Permite obtener un resumen de nuestro data Frame de trabajo, es un compendio de información importante.

DESCRIBE(). Nos da un resumen de los datos del DataFrame.

Ejercicio 4.

Crea un DataFrame llamado `datos_clima` usando Pandas para cargar un archivo CSV llamado "clima.csv" que contenga datos sobre ciudad, temperaturas y precipitaciones en diferentes ciudades (debe crear el archivo csv en excel o un block de notas). Asegúrate de importar Pandas antes de intentar cargar el archivo.

Ejercicio 5.

Utiliza el DataFrame `datos_clima` creado en el ejercicio anterior. Aplica el método `describe()` para explorar la estructura del DataFrame y obtener un resumen estadístico de las columnas numéricas.

Asegurese de almacenar el resultado en una variable llamada `describe_df`

Ejercicio 6.

Utiliza el DataFrame clima creado en el ejercicio anterior.
Aplica los métodos correspondientes para:

Extraer exactamente la primera fila de 'clima'. Almacena el contenido en una variable llamada inicio

Extraer exactamente la última fila de 'clima'. Almacena el contenido en una variable llamada final

Manejo de series.

(Precipitaciones.csv)

Podemos de un data frame hacer lo siguiente:

- Extraer series.
- Uso de `head()` y `tail()` en las series.
- Convertir listas en series.
- Cambiar el índice a los datos de una serie (índice personalizado).
- Identificar los tipos de datos dentro de la serie.
- Convertir un diccionario a una serie.

Ejercicio 07

Crea una serie en Pandas a partir de la siguiente lista de números: [4, 7, -5, 3]. Llama a esta serie `mi_serie`. Imprime `mi_serie` para ver el resultado.

Ejercicio 08

Crea una serie en Pandas a partir de la lista [10, 20, 30, 40] con índices personalizados ['a', 'b', 'c', 'd']. Llama a esta serie `serie_con_indices`.

- Crea una variable llamada `valor_c` donde almacenes el valor asociado al índice 'c'
- Crea una función llamada `imprimir_valor_c` que se capaz de mostrar el valor asociado al índice 'c'.

Ejercicio 09

Utiliza el diccionario {'a': 30, 'b': 70, 'c': 160, 'd': 50} para crear una serie en Pandas, la cual llamarás `serie_desde_diccionario`.

- Luego, accede a los valores asociados a los índices 'a' y 'd', sumándolos.
- Almacena esta suma en la variable `suma_ad`
- Crea una función llamada: `imprimir_suma_ad()` que se encargue de imprimir el valor de `suma_ad`

Manejo de series.

Operaciones básicas con series.

`Serie[0] = serie[0] + 100`

`Serie = Serie + valor`

`Serie = Serie - valor`

`Serie = Serie * valor`

`Serie = Serie ** exponente`

Ejercicio 10.

Crea dos series de pandas utilizando listas de Python.

Puedes crear ambas series con los números enteros de tu elección solamente asegurate de que la serie 1 y la serie 2 las almacenes en variables nombradas: serie1 y serie2 respectivamente.

Luego, suma ambas series y asigna el resultado a una variable llamada serie_sumada. Imprime el resultado de serie_sumada.

Ejercicio 11.

Dada una serie de pandas llamada `serie_numerica` que contenga los números de tu preferencia.

Realiza las siguientes operaciones matemáticas y guarda los resultados en variables separadas:

Multiplica `serie_numerica` por 2 y guarda el resultado en `serie_doble`.

Divide `serie_numerica` por 10 y guarda el resultado en `serie_dividida`.

Imprime los resultados de `serie_doble` y `serie_dividida`.

Ejercicio 12

Considera una serie de pandas llamada `serie_grande` que contiene los números del 1 al 10 o mayor al 10. Obten su ítem 9 y házla su raíz cuadrada.

Imprime el resultado.

Limpieza de datos.

Este proceso se debe realizar antes de poder llevar a cabo el análisis de datos para no contaminar los resultados, se deben identificar valores faltante o nulos, manejo de tipos de datos inapropiados, eliminación de duplicados.

```
data = { "id_producto": [1001, 1002, 1003, 1003],  
        "Cantidad_vendida" : [30, None, 25, 25],  
        "Precio" : [20.5, 15.0, None, 22.5]}
```

Para un resumen rápido podemos usar `dataFrame.info( )`

Revisión de valores nulos.

Para tablas pequeñas.

```
print(dataFrame.isnull())
```

Para tablas grandes

```
print(dataFrame.isnull().sum())
```

¿Qué decisión tomar?

No hay una solución única, la solución depende del caso, hay ocasiones dónde podemos asignar de acuerdo al objetivo del estudio, un valor de cero, o tal vez un valor promedio, quizá eliminar el renglón si es que el estudio no es tan exacto.



Eliminar los valores faltantes

Eliminar valores faltantes.

```
nuevo_df = dataframe.dropna( )
```

Rellenar con ceros o valor promedio.

```
valores_nuevos = {"Cantidad_vendida" : 0, "Precio" :  
df["Precio"].mean() }
```

```
df_rellenados = df.fillna(valores_nuevos)
```

Cambiando el tipo de dato

Esto aplica cuando el tipo de dato no es adecuado, por ejemplo si lo analizamos, en nuestro data frame

	id_producto	Cantidad_vendida	Precio
0	1001	30.0	20.500000
1	1002	0.0	15.000000
2	1003	25.0	19.333333
3	1003	25.0	22.500000

La cantidad vendida siempre debe ser entero y no un valor flotante, por lo que para cambiar el tipo de dato hacemos lo siguiente:

```
dataFrame["Cantidad_vendida"] = dataFrame["Cantidad_vendida"].astype(int)
```

Eliminar duplicados.

Elimina registros que en su totalidad estén duplicados, en caso de existir diferencia en alguno de los valores no los considera idénticos y no elimina el registro.

En caso de eliminar los duplicados deja solo un original.

```
Newdf = df.drop_duplicates( )
```

Solo elimina los registros que son idénticos en su totalidad entre todos los valores!

Eliminar duplicados sobre una serie como criterio de referencia.

Elimina registros duplicados con base a una serie como referencia.

```
df_unicos = df.drop_duplicates(subset="serie_criterio")
```

Ejercicio 13

Dada una tabla (Diccionario) de ventas que contiene información sobre productos vendidos, incluyendo ID, Producto, Cantidad y Precio,

```
data = {  
    'ID': [1, 2, 3, 4, 5],  
    'Producto': ['Producto A', 'Producto B', None, 'Producto D', 'Producto E'],  
    'Cantidad': [10, 20, 30, None, 50],  
    'Precio': [100, 200, 300, 400, None]  
}
```

Escribe un código en Python usando Pandas para contar los valores nulos por columnas.

Ejercicio 14

El conjunto de datos proporcionado tiene algunas entradas duplicadas.

```
data = {  
    'ID': [1, 2, 3, 4, 1],  
    'Producto': ['Producto A', 'Producto B', 'Producto C', 'Producto D',  
    'Producto A'],  
    'Cantidad': [10, 20, 30, 40, 50],  
    'Precio': [100, 200, 300, 400, 100]  
}
```

Escribe un script en Python usando Pandas para eliminar las filas duplicadas en el conjunto de datos. Almacena el resultado en una variable llamada: `df_sin_duplicados`

Ejercicio 15

En tu tabla de ventas, la columna "Precio" tiene algunos valores nulos.

```
data = {  
    'ID': [1, 2, 3, 4],  
    'Producto': ['Producto A', 'Producto B', 'Producto C', 'Producto D'],  
    'Cantidad': [10, 20, 30, 40],  
    'Precio': [100, None, 300, None]  
}
```

Escribe un código en Python usando Pandas para reemplazar los valores nulos en la columna Precio por el promedio de los precios no nulos de esa columna.

Filtrado de series.

El filtrado de series es como si le hiciéramos preguntas específicas a nuestros datos y este nos respondiera.

`filtro = serie criterio`

`serie_filtrada = serie[filtro]`

EN RESUMEN! Filtrar datos es seleccionar datos que cumplan con ciertos criterios.

Ejercicio 16

Crea una serie de Pandas llamada: `serie_numeros` que contenga los números del 1 al 20.

Luego, escribe un código que filtre y muestre solo aquellos números que sean mayores que 10.

Utiliza una variable llamada `filtro` para almacenar la serie filtrada.

Ejercicio 17

Dada una serie de Pandas que contiene los siguientes valores: [18, 22, 7, 9, 15, 8], filtra y muestra solo aquellos valores que sean pares.

Primero, crea una serie de valores booleanos que represente la condición. Nombra a esta variable como: `condicion_valores_pares`

Luego aplica esta serie para filtrar los valores originales.

Ejercicio 18

Crea una serie de Pandas llamada frutas y que contenga los siguientes elementos: ["manzana", "banana", "cereza", "durazno", "frambuesa"].

Escribe un código que filtre y muestre solo aquellos elementos que contengan la letra "e" en su nombre. Almacena los elementos filtrados en una variable llamada frutas_con_e

Utiliza una condición que aplique un método de strings para lograr este filtrado.

Agregación de series en pandas.

En Pandas, las series (columnas en un DataFrame) ofrecen diversas funciones de agregación para resumir y analizar datos.

- La agregación es un conjunto de herramientas para resumir nuestros datos.
- Básicamente, una agregación es una operación que combina varios valores de datos en un solo valor representativo, como puede ser un promedio o una suma total. Estas funciones siguen la misma lógica de SQL.

Agregación de series en pandas.

En Pandas, las series (columnas en un DataFrame) ofrecen diversas funciones de agregación para resumir y analizar datos.

- `Sum( )` Calcula la suma de los valores de la serie.
- `mean( )` Calcula el promedio o media de los valores.
- `Median( )` Calcula el valor central de la serie.
- `min( )` Devuelve el valor mínimo de la serie.
- `max( )` Devuelve el valor máximo de la serie.
- `count( )` Devuelve la cantidad de valores no nulos en la serie.
- `std( )` Calcula la desviación estándar de los valores.

Ejercicio 19.

Crea una serie de Pandas a partir de la siguiente lista de edades = [23, 30, 26, 27, 22, 24, 25, 28]. Luego, utiliza la función adecuada para calcular el promedio de estas edades. Almacena el promedio en una variable llamada: promedio_edades

Ejercicio 20.

Dada lista con los siguientes valores de ventas diarias de una tienda en una semana:

`ventas = [120, 150, 90, 200, 210, 130, 160]`, realiza las siguientes tareas:

Crea una serie de Pandas con los días de la semana como índice (asumiendo que el primer valor corresponde al Lunes).

Calcula y muestra la suma total de las ventas de la semana. Almacena este valor en una variable llamada: `suma_total_ventas`

Encuentra y muestra el día de mayores ventas. Almacena este valor en una variable llamada: `dia_mayores_ventas`

Calcula y muestra el promedio de ventas de la semana. Almacena este valor en una variable llamada: `promedio_ventas`

Ejercicio 21

Considera una serie de Pandas que representa las ventas de un mes, donde cada valor corresponde a las ventas de un día específico.

Realiza las siguientes operaciones y muestra los resultados empleando los métodos correspondientes:

Calcula el total de ventas del mes. Almacena su valor en una variable llamada: `total_ventas_mes`

Encuentra el día con las ventas más bajas. Almacena su valor en una variable llamada: `dia_ventas_mas_bajas`

Determina el promedio mensual de ventas. Almacena su valor en una variable llamada: `promedio_ventas_me`

```
ventas_mes = pd.Series([220, 235, 260, 213, 202, 298, 265, 198, 220, 230, 190, 215, 275, 222, 218,  
                        245, 233, 210, 290, 210, 215, 220, 225, 230, 245, 250, 260, 270, 280, 295])
```

Examen general.

Objetivo.

Realizar un análisis exploratorio de datos (EDA) sobre un conjunto de datos de medallas olímpicas utilizando Pandas.

Este proyecto le permitirá aplicar los conceptos aprendidos sobre Series, DataFrames, limpieza de datos, operaciones básicas, filtrado y agregación en Pandas.

- Contamos con el archivo medallas.csv, que incluye información sobre las medallas de oro, plata, bronce y el total obtenido por cada país en los Juegos Olímpicos
- Vas a realizar una serie de tareas básicas, que te permitirán responder a las preguntas del siguiente cuestionario:

Examen general.

1. Cargar los Datos: Importar los datos desde el archivo CSV a un DataFrame de Pandas
2. Exploración Inicial: Utilizar métodos básicos para explorar el tamaño, las columnas y los tipos de datos del Data Frame.
3. Limpieza de Datos: Identificar y manejar valores faltantes o incorrectos, especialmente en las columnas de medallas donde los valores faltantes indican cero medallas.
4. Análisis de Medallas por País: Realiza las operaciones que sean necesarias para identificar cuáles fueron los 3 países con más medallas de oro en total (va a necesitar investigar los métodos de data frames para encontrar cuál le permite ordenar los valores de mayor a menor o viceversa).
5. Análisis de Medallas Totales por País: Obtener un data frame que contenga sólo los países que ganaron más de 10 medallas en total.

Examen general.

Una vez que haya realizado estos pasos, realiza el cuestionario, que contiene preguntas que solo podrás responder tras haber explorado los datos siguiendo los pasos anteriores

Cuestionario

¿Cuál era el tipo de dato original (Dtype) de la columna “Bronce”?

int64

float64

object

Cuestionario

¿Cuál es la desviación estándar (std) de la columna "Total"?

18.897617	19.56221	19.56220	22.00554
-----------	----------	----------	----------

Cuestionario

¿Cuál fue el segundo país con más medallas de oro?

Alemania

Estados Unidos

Japón

República popular China

Cuestionario

¿Cuántas medallas en total tuvo el país con más medallas conseguidas?

162

113

102

89

Operaciones con Data Frames

Para esta sección tomar el siguiente diccionario:

```
data = {  
    'Nombre' : ['Ana', 'Luis', 'Carlos', 'Sara'],  
    'Edad' : [25, 30, 22, 27],  
    'Ciudad' : ['Acapulco', 'Chilpancingo', 'Iguala', 'Zihuatanejo'] }
```

Operaciones con Data Frames

Agregar series a nuestro data frame

```
dataFrame[NuevaSerie] = [Valores equivalentes a registros]
```

```
ej. df['Salario'] = [10000, 20000, 25000, 30000]
```

Operaciones con Data Frames

Modificando los datos de cualquier serie.

`dataFrame[Serie] = dataFrame[Serie] Operacion (ej. + 200)`

ej. `df['Salario'] = df['Salario'] + 2000`

Operaciones con Data Frames

Agregar series con datos de otras series

`dataFrame[SerieNueva] = dataFrame[Serie]` Operacion

ej. `df['Edad avanzada'] = df['Edad'] + 50`

Operaciones con Data Frames

Crear una variable del tipo serie con los datos de la serie de nuestro data frame

```
Variable = dataframe[Serie]
```

```
ej. serie_nombres = df['Nombre']
```

Operaciones con Data Frames

Filtrado de series dentro del data frame

Variable = dataframe[Serie]

```
ej. mayores_25 = df [ df ['Edad'] > 25]  
    print(mayores_25)
```

Ejercicio 22

Crea un DataFrame llamado `libros_df` con la siguiente información sobre libros:

Los títulos son: "El Quijote", "Cien años de soledad", "La odisea"

Los autores son "Miguel de Cervantes", "Gabriel García Márquez", "Homero"

Los años de publicación son 1605, 1967, y -800, respectivamente.

Asegúrate de utilizar las columnas `Titulo`, `Autor` y `Año`.

Ejercicio 23

Dado el DataFrame df con columnas Nombre, Edad y Ciudad, escribe un código para filtrar las filas que contienen personas de 25 años o más. Asegurate de guardar en el dataframe filtrado en una variable llamada: df_filtrado

```
data = { 'Nombre' : ['Ana', 'Luis', 'Carlos', 'Sara'],  
         'Edad' : [25, 30, 22, 27],  
         'Ciudad' : ['Madrid', 'Barcelona', 'Valencia', 'Bilbao'] }
```

Ejercicio 24

Partiendo del DataFrame df mencionado anteriormente:

```
data = {  
    'Nombre': ['Ana', 'Luis', 'Carlos', 'Sara'],  
    'Edad': [25, 30, 22, 27],  
    'Ciudad': ['Madrid', 'Barcelona', 'Valencia', 'Bilbao']  
}
```

Realiza las siguientes operaciones:

Crea un dataframe llamado df

Agrega una nueva columna llamada Edad_en_10_años que contenga la edad de las personas dentro de 10 años.

Modifica la columna Ciudad para que todas las ciudades estén en mayúsculas.

Crea una nueva columna Es_Mayor_de_25 que contenga valores booleanos: True si la persona tiene 25 años o más, y False en caso contrario.

Ordenar y agrupar data frames.

Ordenar es organizar la información bajo cierto criterio.

Trabajar con **Top-peliculas.csv**

Ordenar de menor a mayor (ascendente).

```
new_df = df.sort_values(by='rating')
```

Ordenar y agrupar data frames.

Ordenar de mayor a menor (descendente).

```
new_df = df.sort_values(by='rating', ascending=False)
```

Ordenamiento con mas de un criterio.

```
df_ordenado = df.sort_values(by=['rating', 'recaudación(M)'],  
ascending=False)
```

Agrupación de datos group by.

Es reunir datos basados en categorías.

- Queremos obtener el promedio del rating por genero.

```
df_agrupado = df.groupby('género')['rating'].mean()
```

Ejercicio 25.

Crea un DataFrame a partir del siguiente diccionario de datos de películas.

```
datos = {  
    'titulo': ['Película A', 'Película B', 'Película C', 'Película D'],  
    'año': [2001, 2000, 2005, 2010],  
    'rating': [7.2, 6.5, 8.1, 7.5]  
}
```

Luego, ordena este DataFrame por la columna 'año' en orden ascendente. Asegurate de almacenar el diccionario ordenado en una variable llamada: `df_ordenado`

Ejercicio 26

Usando el mismo DataFrame del Ejercicio anterior.

```
datos = {  
    'titulo': ['Pelicula A', 'Pelicula B', 'Pelicula C', 'Pelicula D'],  
    'año': [2001, 2000, 2005, 2010],  
    'rating': [7.2, 6.5, 8.1, 7.5]  
}
```

Ordena las películas primero por 'rating' en orden descendente y luego por 'año' en orden ascendente. Asegurate de almacenar el dataframe ordenado en una variable llamada: df_ordenado

Ejercicio 27

Crea un nuevo DataFrame a partir del siguiente diccionario

```
datos = {  
    'titulo': ['Película A', 'Película B', 'Película C', 'Película D', 'Película E'],  
    'género': ['Acción', 'Drama', 'Acción', 'Comedia', 'Drama'],  
    'rating': [7.2, 8.5, 9.1, 6.5, 7.8]  
}
```

Que contiene datos de películas incluyendo su género.

Luego, agrupa las películas por 'género' y calcula el promedio de 'rating' para cada género. Asegurate de almacenar el grupo y promedio en una variable llamada: `promedio_rating_por_genero`

Fusionar data frames con merge

MERGE.

- Permite combinar valores basándose en los valores de una columna específica. Es común que muchas veces obtengamos datos desde distintas fuentes, pero que esos datos compartan información sobre los mismos elementos.
- Muchas veces los datos pueden provenir de diferentes tablas y que esas tablas proporcionan diferentes datos sobre los mismos elementos, puedes por ejemplo, tener una tabla con los nombres y la información personal de tus clientes y por otro lado, una tabla separada con los pedidos que esos clientes han realizado, son los mismos registros en diferentes tablas que se ocupan de diferentes aspectos del mismo elemento de la realidad

Fusionar data frames con merge

Para el tema de MERGE usar los siguientes data frame:

```
df1 = pd.DataFrame({'ID': [1, 2, 3],  
                    'Nombre': ['Ana', 'Luis', 'Carlos']})
```

```
df2 = pd.DataFrame({'ID':[1, 2, 4],  
                    'Edad': [25, 30, 22]})
```

Fusionar data frames con merge

SOLO LAS INTERSECCIONES DEL CONJUNTO.

```
pandas.merge ( dF1, dF2, on = 'SerieCriterio')
```

```
pandas.merge ( dF1, dF2, on = 'SerieCriterio', how='inner' )
```

how inner por defecto

Fusionar data frames con merge

TODOS LOS VALORES.

- Donde no hay coincidencias rellena los valores con NaN.

```
df_combinado = pd.merge(df1, df2, on='ID', how='outer')
```

Fusionar data frames con merge

CON PRIORIDAD A LA IZQUIERDA.

- LEFT toma todos los valores del data frame de la izquierda y las intersecciones entre el data frame 1 y el data frame 2

```
df_combinado = pd.merge(df1, df2, on='ID', how='left')
```

Fusionar data frames con merge

CON PRIORIDAD A LA DERECHA.

- RIGHT toma todos los valores del data frame de la derecha y las intersecciones entre el data frame 1 y el data frame 2

```
df_combinado = pd.merge(df1, df2, on='ID', how='right')
```

Fusionar data frames con merge

POR REFERENCIA A LOS ÍNDICES.

- Los valores de los data frame los combina por el valor del índice.

```
df_combinado = pd.merge(df1, df2, left_index=True,  
right_index=True )
```

Combinar data frames con join.

JOIN

- Bajo el concepto de JOIN se realiza una unión mediante su índice o de una columna clave, pero no de una columna compartida, solo confiando en que sus índices se corresponden entre sí.

Combinar data frames con join.

Para el tema de JOIN usar el siguiente data frame:

```
df1 = pd.DataFrame({'Salario': [30000, 45000, 30000],  
                    'Antigüedad': [9, 13, 12],},  
                    index = [1, 2, 3])
```

```
df2 = pd.DataFrame({'Ciudad': ['Iguala', 'Acapulco', 'Chilpancingo' ],  
                    'Jerarquía': ['Baja', 'Alta', 'Media']},  
                    index = [1, 2, 4])
```

Combinar data frames con join.

JOIN CARGADO A LA IZQUIERDA (POR DEFECTO).

```
df_unido = df1.join(df2)
```

```
df_unido_left = df1.join(df2, how='left')
```

Combinar data frames con join.

JOIN SOLO INTERSECCIONES.

```
df_unido_left = df1.join(df2, how='inner')
```

Podemos probar right.

Ejercicio 28

Dados dos DataFrames, df_a y df_b. df_a

```
df_a = pd.DataFrame({ 'id': [1, 2, 3], 'Nombre': ['Ana', 'Beto', 'Carla'] })
```

```
df_a.set_index('id', inplace=True)
```

```
df_b = pd.DataFrame({ 'id': [1, 2, 3], 'Edad': [25, 30, 35] })
```

```
df_b.set_index('id', inplace=True)
```

Utiliza el método join() para combinar df_a y df_b en un nuevo DataFrame llamado df_combinado, usando el argumento por defecto de how.

Concatenar data frames

La operación de concatenado es necesaria, debido a que por ejemplo pensemos que nuestras tiendas nos reportan a diario las ventas, como característica las estructuras de los archivos debe ser idéntica, esto es, las mismas columnas, los mismos datos, con esto CONCAT nos va a permitir juntas las tablas en una sola.

Concatenar data frames

Para este ejercicio usar:

```
df1 = pd.DataFrame({'Nombre': ['Juan', 'Gabriela', 'Elena'],  
                    'Edad': [23, 31, 21]})
```

```
df2 = pd.DataFrame({'Nombre': ['Carmela', 'Max', 'Leticia'],  
                    'Edad': [34, 25, 29]})
```

Concatenar data frames

CONCATENACIÓN HORIZONTAL POR COLUMNAS.

```
df_concatenado = pd.concat([df1, df2])
```

VALORES PARA LA CONCATENACIÓN.

axis=0 concatenan de forma vertical por columna (valor por defecto se indique o no)

axis=1 concatenan de forma horizontal por filas

Resetear el índice.

En el ejemplo anterior al ver el resultado de la tabla concatenada podemos observar que se mantienen los índices originales.

	Nombre	Edad
0	Juan	23
1	Gabriela	31
2	Elena	21
0	Carmela	34
1	Max	25
2	Leticia	29

Resetear el índice.

Para evitar esto podemos resetear los valores para que sean únicos e irrepetibles, por defecto existe un parámetro `ignore_index=False`, para resetear los valores de los índices simplemente lo cambiamos `True`.

```
df_concatenado = pd.concat([df1, df2], ignore_index=True)
```

Agregar marcas para identificar cada grupo

Podemos indicar que nos muestre una etiqueta para identificar a que data frame pertenece cada grupo de datos, de esta manera podemos identificar a qué grupo de datos pertenecen los valores concatenados.

Las etiquetas pueden ser cualquier valor, solo son para asociar los grupos de valores

- `df_concatenado = pd.concat([df1, df2], keys=['df1', 'df2'])`

Leer archivos con pandas

Podemos leer archivos mediante pandas, no precisamente solo archivos CSV, en el siguiente ejemplo se leerá un archivo de Excel

```
df1 = pd.read_excel(ruta_excel)
```

```
df2 = pd.read_xml(ruta_xml)
```

(Compras_desde_ads.xlsx, Valores de acciones.xml)

Escribir archivos con pandas

De igual manera podemos también escribir archivos, en el siguiente ejemplo se escribira un archivo en excel.

`df.to_excel(ruta)`

Cuando usamos la instrucción anterior se escribe el índice, para evitar

`df.to_excel(ruta, index=False)`

Escribir archivos con pandas

Para escribir archivos en formato CSV.

`df.to_csv(ruta, index=False)`

Ejercicio 29

Dado el siguiente diccionario que contiene datos de ventas diarias de una tienda:

```
Data = { 'Fecha': ['2024-03-19', '2024-03-20', '2024-03-21', '2024-03-22', '2024-03-23', '2024-03-24'],  
        'Producto': ['Manzanas', 'Peras', 'Naranjas', 'Plátanos', 'Uvas', 'Melocotones'],  
        'Cantidad': [23, 15, 18, 30, 8, 20],  
        'Precio': [1.2, 1.5, 1.0, 0.8, 2.0, 1.7] }
```

Realiza los siguientes pasos:

Crea un DataFrame nombrado `df_ventas` donde almacenes la información antes proporcionada.

Gusada este DataFrame en un archivo llamado `ventas1.xlsx`

Asegúrate de no incluir el índice del DataFrame en el archivo Excel.

Ejercicio 30

Dado el siguiente diccionario que contiene datos de ventas diarias de una tienda:

```
Data = { 'Fecha': ['2024-03-19', '2024-03-20', '2024-03-21', '2024-03-22', '2024-03-23', '2024-03-24'],  
        'Producto': ['Manzanas', 'Peras', 'Naranjas', 'Plátanos', 'Uvas', 'Melocotones'],  
        'Cantidad': [23, 15, 18, 30, 8, 20],  
        'Precio': [1.2, 1.5, 1.0, 0.8, 2.0, 1.7] }
```

Realiza los siguientes pasos:

Crea un DataFrame nombrado `df_ventas` donde almacenes la información antes proporcionada.

Guarde este DataFrame en un archivo llamado 'ventas2.xlsx'

Asegúrate de no incluir el índice del DataFrame en el archivo Excel.

Ejercicio 31

Dado un diccionario con datos de ventas diarias de una tienda como sigue:

```
data = { 'Fecha': ['2024-03-19', '2024-03-20', '2024-03-21', '2024-03-22', '2024-03-23', '2024-03-24'],  
        'Producto': ['Manzanas', 'Peras', 'Naranjas', 'Plátanos', 'Uvas', 'Melocotones'],  
        'Cantidad': [23, 15, 18, 30, 8, 20],  
        'Precio': [1.2, 1.5, 1.0, 0.8, 2.0, 1.7] }
```

Tu tarea consiste en realizar las siguientes acciones:

Creación de DataFrame: Genera un DataFrame denominado `df_ventas` utilizando la información provista en el diccionario `data`.

Cálculo de Precio Total: Añade al DataFrame `df_ventas` una nueva columna llamada `Precio Total`. Esta columna debe contener el precio total de cada venta, calculado como el producto de la cantidad vendida por el precio unitario de cada producto.

Guardado de Datos en CSV: Exporta el DataFrame `df_ventas` a un archivo CSV. Nombra el archivo como `ventas3.csv`. Asegúrate de que el archivo CSV generado no incluya el índice del DataFrame.

Fin parte dos.